

TECH

TESTING

UNIT / INTEGRATION E2E / PLAYWRIGHT

E2E / PLAYWRIGHT

25 ВОПРОСОВ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК playwright.dev

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	ЧТО ТЕСТИРОВАТЬ E2E		LOCATORS	get	ЧТО ТЕСТИРОВАТЬ E2E	LOCATORS	get	
	ByRole				ByRole			
	Q1 · Q2 · Q3 · Q4							
ДЕНЬ 2 СРЕДНИЙ	AUTO-WAITING	SLEEPS	RETRIES	F	AUTO-WAITING	SLEEPS	RETRIES	F
	LAKY DETECT				LAKY DETECT			
	Q5 · Q6 · Q7 · Q8							
ДЕНЬ 3 ТЯЖЁЛЫЙ	AUTH	STORAGE	STATE	ISOLATION	AUTH	STORAGE	STATE	ISOLATION
	NETWORK	FAIL	SLOW		NETWORK	FAIL	SLOW	
	Q9 · Q10 · Q11 · Q12 · Q13							
ДЕНЬ 4 СРЕДНИЙ	PERMISSIONS	FLAGS	VISUAL	REGRESS	PERMISSIONS	FLAGS	VISUAL	REGRESS
	ION			ION				
	Q14 · Q15 · Q16							
ДЕНЬ 5 ТЯЖЁЛЫЙ	RESPONSIVE	REAL-TIME	CI	SUITE	RESPONSIVE	REAL-TIME	CI	SUITE
	TIME				TIME			
	Q17 · Q18 · Q19 · Q20							
ДЕНЬ 6 СРЕДНИЙ	SMOKE	TEST DATA	CLEANUP		SMOKE	TEST DATA	CLEANUP	
	Q21 · Q22 · Q23							
ДЕНЬ 7 СРЕДНИЙ	ANALYTICS	OBSERVABILITY			ANALYTICS	OBSERVABILITY		
	Q24 · Q25							

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 4 ВОПРОСА

ЧТО ТЕСТИРОВАТЬ E2E LOCATORS getByRole

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ЧТО ТЕСТИРОВАТЬ E2E LOCATORS getByRole

ВОПРОСЫ ДНЯ

- Q01 Что тестировать в Playwright?
- Q02 Что не стоит тестировать в Playwright?
- Q03 Как выбирать locators?
- Q04 Почему getByRole часто лучше CSS selector?

Что тестировать в Playwright?

ОПИСАНИЕ

Critical user paths: login, signup, key-feature flows, checkout, search. Cross-browser smoke. Не – каждый компонент (это integration в jest).

КАК РАБОТАЕТ

- 01 Critical user paths.
- 02 Cross-browser smoke.
- 03 End-to-end intentions.
- 04 Real browser behavior.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production releases.
- 02 Critical features.

ПОДВОДНЫЕ КАМНИ

- 01 Каждый button – overkill.
- 02 E2E на implementation – fragile.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое critical path?
- 02 Чем e2e от integration отличается?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices>

ВОПРОС Q02

Что не стоит тестировать в Playwright?

ОПИСАНИЕ

Каждое UI-состояние, каждую кнопку, бизнес-логику. Это для unit/integration. E2E медленны, лучше – high-value scenarios.

КАК РАБОТАЕТ

- 01 Не каждое state.
- 02 Не каждую кнопку.
- 03 Не бизнес-логику.
- 04 Только critical scenarios.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Test strategy review.

ПОДВОДНЫЕ КАМНИ

- 01 Slow CI с 1000 e2e tests.
- 02 Flaky overload.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое test pyramid?
- 02 Чем integration лучше для UI?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices>

Как выбирать locators?

ОПИСАНИЕ

Приоритет: getByRole, getByLabel, getByPlaceholder, getByText, getByTestId. Role-based – самые user-centric и стабильные.

КАК РАБОТАЕТ

- 01 Priority: role > label > placeholder > text > testid.
 - 02 Auto-waiting встроен.
 - 03 Match accessibility tree.
 - 04 Avoid CSS selectors.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любые e2e tests.
-

ПОДВОДНЫЕ КАМНИ

- 01 data-testid вместо role – fragile + non-user.
 - 02 CSS selectors break при refactor.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое accessibility tree?
 - 02 Чем chain-locators полезны?
-

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/locators>

ВОПРОС Q04

Почему getByRole часто лучше CSS selector?

ОПИСАНИЕ

Отражает как user (и SR) воспринимает UI. Стабилен к refactor (class изменился – role нет). Auto-waiting встроен. Также tests доступность.

КАК РАБОТАЕТ

- 01 User-centric matching.
- 02 Refactor-resistant.
- 03 Auto-waiting.
- 04 Doubles as ally test.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой e2e.

ПОДВОДНЫЕ КАМНИ

- 01 CSS selector fragile.
- 02 Class names меняются.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое accessibility tree?
- 02 Чем accessible name полезен?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/locators#locate-by-role>

AUTO-WAITING

SLEEPS

RETRIES

FLAKY DETECT

ЧТО УЧИМ В ЭТОТ ДЕНЬ

AUTO-WAITING

SLEEPS

RETRIES

FLAKY DETECT

ВОПРОСЫ ДНЯ

Q05 Что такое auto-waiting?

Q06 Почему explicit sleeps плохие?

Q07 Как работать с retries?

Q08 Как отличать flaky test от product bug?

ВОПРОС Q05

Что такое auto-waiting?

ОПИСАНИЕ

Playwright ждёт, пока element actionable (visible, enabled, stable) перед action. Нет нужды в explicit waits. Timeout default 30s, кастомизируемый.

КАК РАБОТАЕТ

- 01 Wait for actionability.
- 02 No manual sleep.
- 03 Default 30s timeout.
- 04 Configurable per action.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любой e2e action.

ПОДВОДНЫЕ КАМНИ

- 01 Без понимания – добавляют sleep.
- 02 Misuse {force: true}.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое actionability?
- 02 Чем strict mode полезен?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/actionability>

Почему `explicit sleeps` плохие?

ОПИСАНИЕ

`page.waitForTimeout(N)` – недетерминирован: `too small` → flaky, `too large` → slow CI. `Auto-waiting` и `waitFor*` делают это правильно. `Sleep` – code smell.

КАК РАБОТАЕТ

- 01 Non-deterministic.
- 02 Slow CI.
- 03 Flaky cause.
- 04 Anti-pattern.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Code review e2e.

ПОДВОДНЫЕ КАМНИ

- 01 `sleep` везде → slow + flaky.
- 02 'Just add 5s' culture.

МОГУТ ДОСПРОСИТЬ

- 01 Чем `waitForLoadState` полезен?
- 02 Что такое `expect.toHaveText`?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices#dont-use-page-waitfortimeout-in-production>

ВОПРОС Q07

Как работать с retries?

ОПИСАНИЕ

playwright.config.retries: 2 – flaky tests retry. Investigate каждый flaky test. Не как mask для bugs. Failed retries – серьёзная проблема.

КАК РАБОТАЕТ

- 01 Config retries (CI: 2, local: 0).
- 02 Investigate root cause.
- 03 Не mask bugs.
- 04 Track flaky rate.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 CI configuration.

ПОДВОДНЫЕ КАМНИ

- 01 Retry forever → masks bugs.
- 02 Без investigation – flaky накопится.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flaky-test rate?
- 02 Чем local от CI retries отличаются?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/test-retries>

Как отличать flaky test от product bug?

ОПИСАНИЕ

Flaky – иногда red, иногда green без code change. Bug – consistent red. Run N times → если 50/50 → flaky, если 100% red → bug. Investigate flaky немедленно.

КАК РАБОТАЕТ

- 01 Run N times.
- 02 Inconsistent → flaky.
- 03 Consistent red → bug.
- 04 Investigate immediately.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 CI debugging.
- 02 Test maintenance.

ПОДВОДНЫЕ КАМНИ

- 01 Игнор flaky – накапливается, тесты становятся бесполезны.
- 02 Mark as bug без investigation.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flake-test pattern?
- 02 Чем investigation полезна?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices>

ДЕНЬ 3

НАГРУЗКА

ТЯЖЁЛЫЙ

5 ВОПРОСОВ

AUTH

STORAGE STATE

ISOLATION

NETWORK FAIL

SLOW

ЧТО УЧИМ В ЭТОТ ДЕНЬ

AUTH

STORAGE STATE

ISOLATION

NETWORK FAIL

SLOW

ВОПРОСЫ ДНЯ

Q09 Как тестировать auth?

Q10 Как шарить storage state?

Q11 Как изолировать тесты?

Q12 Как тестировать network failures?

Q13 Как тестировать slow network?

Как тестировать auth?

ОПИСАНИЕ

Login один раз, save storage-state (cookies + localStorage), reuse в subsequent tests через context.storageState. Faster – не login per test.

КАК РАБОТАЕТ

- 01 Login + saveStorageState.
- 02 Reuse via storageState option.
- 03 Per-role storage states.
- 04 Setup в global-setup.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Auth-protected tests.
- 02 Multi-role testing.

ПОДВОДНЫЕ КАМНИ

- 01 Login per test – slow.
- 02 Stale storage-state после auth-changes.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое storageState?
- 02 Чем global-setup полезен?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/auth>

ВОПРОС Q10

Как шарить storage state?

ОПИСАНИЕ

Save через `context.storageState({ path: 'auth.json' })`. Use в config: `testOptions.storageState`. Per-role: separate files (admin.json, user.json).

КАК РАБОТАЕТ

- 01 `context.storageState({ path })`.
- 02 `testOptions.storageState`.
- 03 Per-role files.
- 04 Refresh periodically.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Multi-role tests.
- 02 CI optimization.

ПОДВОДНЫЕ КАМНИ

- 01 Stale auth.json.
- 02 Tests modify shared state.

МОГУТ ДОСПРОСИТЬ

- 01 Чем per-role storage полезен?
- 02 Что такое global-setup?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/auth#reuse-signed-in-state>

Как изолировать тесты?

ОПИСАНИЕ

Каждый test в отдельном context (default). Per-test data (создавать unique). Avoid shared mutable state. Cleanup после теста (если что-то persist).

КАК РАБОТАЕТ

- 01 Per-test context.
- 02 Unique test data.
- 03 No shared mutable.
- 04 Cleanup hooks.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая e2e suite.

ПОДВОДНЫЕ КАМНИ

- 01 Shared cookies между тестами.
- 02 Tests modify shared DB.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое test-isolation?
- 02 Чем worker-scoped fixture полезен?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/test-isolation>

ВОПРОС Q12

Как тестировать network failures?

ОПИСАНИЕ

page.route() – intercept requests, return error/timeout. Test: UI shows error, retry works, fallback UI. Test offline через context.setOffline.

КАК РАБОТАЕТ

- 01 page.route() для intercept.
- 02 abort() / fulfill error.
- 03 setOffline для offline.
- 04 Test error UI + retry.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Resilient apps.
- 02 PWA.

ПОДВОДНЫЕ КАМНИ

- 01 Без mock – depends на real network.
- 02 Skip error-UI tests.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое page.route?
- 02 Чем offline-mode полезен?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/network>

Как тестировать slow network?

ОПИСАНИЕ

page.route с delay перед fulfill. Или CDP-flag для throttling. Assert: skeleton-UI / loading state visible, app не падает.

КАК РАБОТАЕТ

- 01 page.route + setTimeout.
- 02 CDP throttling (Chromium).
- 03 Test skeleton / loading.
- 04 Test long-loading message.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Mobile-friendly tests.
- 02 Performance audits.

ПОДВОДНЫЕ КАМНИ

- 01 Real-network slow → flaky.
- 02 Skip без long-loading test.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое CDP?
- 02 Чем skeleton от spinner лучше?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/network#modify-responses>

ДЕНЬ 4 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

PERMISSIONS

FLAGS

VISUAL REGRESSION

ЧТО УЧИМ В ЭТОТ ДЕНЬ

PERMISSIONS FLAGS VISUAL REGRESSION

ВОПРОСЫ ДНЯ

Q14 Как тестировать permissions?

Q15 Как тестировать feature flags?

Q16 Как тестировать visual regressions?

Как тестировать permissions?

ОПИСАНИЕ

Per-role storageState (admin / user / guest). Login as role, assert UI matches expected permissions. Test denied actions return 403 / hidden UI.

КАК РАБОТАЕТ

- 01 Per-role storageState.
- 02 Login as role.
- 03 Assert UI matches role.
- 04 Test denied returns 403/hidden.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 RBAC apps.
- 02 B2B-инструменты.

ПОДВОДНЫЕ КАМНИ

- 01 Single-role testing.
- 02 Без forbidden-action tests.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое RBAC?
- 02 Чем per-role storage полезен?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/auth>

ВОПРОС Q15

Как тестировать feature flags?

ОПИСАНИЕ

Mock flag-provider response через `page.route`. Or set `localStorage` / `cookie` перед `navigation`. Test обе ветки (flag on / off).

КАК РАБОТАЕТ

- 01 Mock provider response.
- 02 Set `localStorage` перед `goto`.
- 03 Test обе ветки.
- 04 Per-flag fixture.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 A/B testing.
- 02 Gradual rollout.

ПОДВОДНЫЕ КАМНИ

- 01 Test только default state.
- 02 Real flag-provider – flaky.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое flag-fixture?
- 02 Чем mock от real provider лучше?

ПОЛНАЯ СТАТЬЯ

<https://docs.growthbook.io/>

Как тестировать visual regressions?

ОПИСАНИЕ

page.screenshot + toHaveScreenshot (Playwright snapshot). Per-component / per-page. Cross-browser screenshot. Update via --update-snapshots.

КАК РАБОТАЕТ

- 01 page.screenshot + toHaveScreenshot.
- 02 Per-component / per-page.
- 03 Cross-browser support.
- 04 --update-snapshots flag.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 DS components.
- 02 UI-критичные.

ПОДВОДНЫЕ КАМНИ

- 01 Font rendering между OS.
- 02 Слишком много snapshots → slow.

МОГУТ ДОСПРОСИТЬ

- 01 Чем Chromatic полезен?
- 02 Что такое pixel diff?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/test-snapshots>

ДЕНЬ 5

НАГРУЗКА

ТЯЖЁЛЫЙ

4 ВОПРОСА

RESPONSIVE**REAL-TIME****CI****SUITE TIME**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

RESPONSIVE REAL-TIME CI SUITE TIME

ВОПРОСЫ ДНЯ

Q17 Как тестировать responsive UI?

Q18 Как тестировать real-time UI?

Q19 Как запускать e2e в CI?

Q20 Как уменьшить время e2e suite?

Как тестировать responsive UI?

ОПИСАНИЕ

page.setViewportSize per test или devices preset. Test mobile / tablet / desktop.
Assert mobile-only UI / desktop-only.

КАК РАБОТАЕТ

- 01 page.setViewportSize().
- 02 devices['iPhone 12'] presets.
- 03 Test mobile / tablet / desktop.
- 04 Per-viewport assertions.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Responsive apps.
- 02 Mobile-friendly products.

ПОДВОДНЫЕ КАМНИ

- 01 Только desktop tests.
- 02 Без mobile-specific assertions.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое devices в Playwright?
- 02 Чем emulation от real device отличается?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/emulation#devices>

ВОПРОС Q18

Как тестировать real-time UI?

ОПИСАНИЕ

Mock WS через page.route с websocket-handlers. Trigger events, assert UI updates. Or use real test-server. Test reconnect / stale.

КАК РАБОТАЕТ

- 01 MSW WS handlers.
- 02 Custom WS test-server.
- 03 Trigger events, assert UI.
- 04 Test reconnect / stale.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Trading UIs.
- 02 Chat apps.

ПОДВОДНЫЕ КАМНИ

- 01 Real WS → slow + flaky.
- 02 Без timing-control – race.

МОГУТ ДОСПРОСИТЬ

- 01 Чем MSW WS полезен?
- 02 Что такое test-server?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/network#websockets>

Как запускать e2e в CI?

ОПИСАНИЕ

Headless. Sharding (split tests across N runners). Caching browsers (npx playwright install). Trace-on-failure для debug. Reporter (HTML / JUnit).

КАК РАБОТАЕТ

- 01 Headless mode.
- 02 Sharding tests.
- 03 Cache browsers.
- 04 Trace on failure.
- 05 HTML / JUnit reporter.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 GitHub Actions / GitLab CI.

ПОДВОДНЫЕ КАМНИ

- 01 Без sharding – slow.
- 02 Без trace – невозможно debug.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое sharding?
- 02 Чем trace от video лучше?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/ci>

ВОПРОС Q20

Как уменьшить время e2e suite?

ОПИСАНИЕ

Sharding (parallel CI). Test isolation parallel within shard. Avoid login per test (storageState). Avoid screenshots без надобности. Trim slow tests.

КАК РАБОТАЕТ

- 01 Sharding.
- 02 Parallel within shard.
- 03 StorageState for auth.
- 04 Selective screenshots.
- 05 Trim slow tests.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Long e2e suites.

ПОДВОДНЫЕ КАМНИ

- 01 Single-runner – long CI.
- 02 Login per test – overhead.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое test sharding?
- 02 Как identify slow tests?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/test-sharding>

SMOKE

TEST DATA

CLEANUP

ЧТО УЧИМ В ЭТОТ ДЕНЬ

SMOKE

TEST DATA

CLEANUP

ВОПРОСЫ ДНЯ

Q21 Какие smoke tests должны быть перед production?

Q22 Как проектировать test data?

Q23 Как чистить test data?

ВОПРОС Q21

Какие smoke tests должны быть перед production?

ОПИСАНИЕ

Critical paths: login, main feature, checkout/order, key dashboard. Покрытие < 30 minutes. Запускаются на каждый PR + post-deploy.

КАК РАБОТАЕТ

- 01 Login flow.
- 02 Main feature.
- 03 Checkout / order.
- 04 Key dashboard.
- 05 < 30 min total.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Pre-production gate.
- 02 Post-deploy validation.

ПОДВОДНЫЕ КАМНИ

- 01 Slow smoke = не запускают.
- 02 Skip-критичные пути.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое critical path?
- 02 Чем smoke от full отличается?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/best-practices>

Как проектировать test data?

ОПИСАНИЕ

Per-test unique (uuid в email). Factories / fixtures (test-user-builder). Avoid shared mutable. Use dedicated test-DB / namespace.

КАК РАБОТАЕТ

- 01 Unique per test (uuid).
- 02 Factories / builders.
- 03 Dedicated test-namespace.
- 04 Avoid shared state.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Любая e2e suite.

ПОДВОДНЫЕ КАМНИ

- 01 Hardcoded user 'test@test' – race condition.
- 02 Shared test-DB → flaky.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое test-builder?
- 02 Чем factory полезна?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/test-fixtures>

ВОПРОС Q23

Как чистить test data?

ОПИСАНИЕ

afterEach hook deletes created entities. Or test-namespace cleanup periodically. Or transactional tests (rollback). Avoid forgetting.

КАК РАБОТАЕТ

- 01 afterEach delete.
- 02 Periodic namespace cleanup.
- 03 Transactional tests.
- 04 Schedule cleanup CI job.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Long-running test infra.

ПОДВОДНЫЕ КАМНИ

- 01 Forgetting cleanup → polluted DB.
- 02 Cleanup ломает parallel tests.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое test fixture cleanup?
- 02 Чем transactional тесты полезны?

ПОЛНАЯ СТАТЬЯ

<https://playwright.dev/docs/test-fixtures>

ANALYTICS

OBSERVABILITY

ЧТО УЧИМ В ЭТОТ ДЕНЬ

ANALYTICS

OBSERVABILITY

ВОПРОСЫ ДНЯ

Q24 Как тестировать analytics events?

Q25 Как тестировать frontend observability?

В О П Р О С Q 2 4

Как тестировать analytics events?

ОПИСАНИЕ

page.route intercept analytics endpoint. Assert events были sent с correct payload. Avoid testing analytics-internal.

КАК РАБОТАЕТ

- 01 page.route analytics.
- 02 Assert events sent.
- 03 Validate payload.
- 04 Mock provider response.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Analytics-критичные продукты.

ПОДВОДНЫЕ КАМНИ

- 01 Real analytics → noise / cost.
- 02 Без assertion – забывают track.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое event tracking?
- 02 Чем sendBeacon от fetch отличается?

ПОЛНАЯ СТАТЬЯ

<https://posthog.com/docs>

Как тестировать frontend observability?

ОПИСАНИЕ

Trigger error → assert Sentry-mock получил event. page.route Sentry endpoint. Verify breadcrumbs, error-context. Test feature-flag toggling tracked.

КАК РАБОТАЕТ

- 01 page.route Sentry.
- 02 Assert event sent.
- 03 Verify breadcrumbs.
- 04 Test flag-tracking.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Production observability.

ПОДВОДНЫЕ КАМНИ

- 01 Real Sentry → noise.
- 02 Skip observability tests → broken alerts.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое breadcrumb?
- 02 Чем mock Sentry полезен?

ПОЛНАЯ СТАТЬЯ

<https://docs.sentry.io/>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ [] Описать что тестировать e2e и locators (Q1-Q4)
- ☐ [] Описать auto-waiting / sleeps / retries / flaky (Q5-Q8)
- ☐ [] Auth / storage state / isolation / network / slow (Q9-Q13)
- ☐ [] Permissions / flags / visual regression (Q14-Q16)
- ☐ [] Responsive / real-time / CI / suite-time (Q17-Q20)
- ☐ [] Smoke / test data / cleanup (Q21-Q23)
- ☐ [] Analytics / observability (Q24, Q25)

EVENT LOOP / ASYNC МЕТОДИЧКА v1
МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru